

SignalR + Notification Errors

How We Debugged & Fixed Them

TaskFlow (ASP.NET Core + Next.js + RTK Query + SignalR)

Real production-style debugging story for interviews — problem, root cause, fix, and talking points.

1. Problem Statement (How to explain in interview)

After adding real-time task-assignment notifications with SignalR, the UI started throwing Next.js error overlays when opening the notification bell or clicking a notification. It looked like a framework crash, but the app was mostly fine — the hard part was separating **real bugs** from **dev-only noise**.

Errors we saw in console / overlay:

- **Error A:** Failed to start the connection: The connection was stopped during negotiation (SignalR)
- **Error B:** Cannot refetch a query that has not been started yet (RTK Query) at NotificationBell.tsx
- **Error C (noise):** Hydration mismatch with cz-shortcut-listen — browser extension, not our app

2. Why this felt hard

Three layers failed at once: real-time transport (SignalR), client data layer (RTK Query), and Next.js Dev overlay (which turns `console.error` into a full-screen error). If you only chase SignalR, you miss the RTK bug. If you only silence logs, the real refetch crash remains.

3. Debugging approach (interview-friendly steps)

- 1 **Reproduce with evidence:** open DevTools → Console + Network. Note exact stack frames (NotificationBell.tsx, SignalR negotiate).
- 2 **Verify backend independently:** login API OK, hub negotiate returns 401 without token / works with JWT → hub itself is alive.
- 3 **Split errors:** one stack from RTK Query, one from SignalR logger. Fix deterministic app bugs first.
- 4 **Check React Strict Mode:** in Next.js/React 18+, effects mount → cleanup → remount in dev. That aborts in-flight SignalR negotiate.
- 5 **Ignore false positives:** hydration attrs injected by extensions are not product bugs.

4. Root Cause Deep Dive

4.1 RTK Query: refetch on a skipped query

Notification list query used `skip: !open`. On bell click we called `refetchList()` **before** the query was allowed to start. RTK throws: *Cannot refetch a query that has not been started yet*. This was a real uncaught UI bug.

Bad pattern:

```
setOpen(true); refetchList(); // query still skipped / not started
```

Fix idea:

```
Just setOpen(true). When skip becomes false, RTK auto-fetches. Never refetch a skipped query.
```

4.2 SignalR: stopped during negotiation

In development, React Strict Mode runs effects twice. Flow: **start()** begins negotiate → cleanup calls **stop()** → negotiate aborts → client reports *connection was stopped during negotiation*. SignalR JS logs this with `console.error`, and Next.js DevTools overlays it as a scary Console Error — even if we catch the promise.

4.3 Next.js overlay amplifies noise

Next.js intercepts console errors in development. Library-level SignalR logging looked like an application crash. Solution: reduce SignalR log level / use a shared connection so Strict Mode remounts do not tear down negotiation mid-flight.

5. Final Solution Architecture

A) NotificationBell (RTK)

- Keep unread-count query always active (badge).
- Keep notifications list query skip: !open.
- On toggle: only flip open — no manual refetch of skipped query.

B) Shared SignalR singleton

- One module-level HubConnection (not created/destroyed on every React effect).
- Subscribers register handlers; Strict Mode remount only adds/removes handlers.
- `configureLogging(LogLevel.None)` to avoid Next overlay spam.
- Treat negotiate-abort messages as benign in catch.

C) Backend SignalR (ASP.NET Core)

- Hub: `/hubs/notifications` with JWT via `access_token` query (WebSockets).
- On connect: join group `user-{userId}`.
- On task assign: save Notifications row, then push `ReceiveNotification` to that group.
- CORS: frontend origin + credentials for SignalR negotiate.

Layer	Symptom	Fix
RTK Query	refetch before start	toggle open only; let skip unlock fetch
SignalR + React	stopped during negotiation	shared connection + no Strict Mode tear-down
Next.js Dev	full-screen Console Error	<code>LogLevel.None</code> / ignore benign aborts
Hydration	<code>cz-shortcut-listen</code>	extension noise; <code>suppressHydrationWarning</code> on body

6. Interview Q&A (memorize these answers)

Q1. What was the actual bug vs the scary overlay?

The real uncaught bug was RTK refetch on a skipped query. The SignalR negotiate message was largely a Strict Mode race amplified by Next.js treating library `console.error` as an overlay.

Q2. Why does React Strict Mode break SignalR start?

Effects run `mount` → `cleanup` → `remount` in development. If `start()` is still negotiating when `cleanup` calls `stop()`, the client throws 'stopped during negotiation'.

Q3. How do you authenticate SignalR with JWT?

Browsers cannot easily set Authorization on WebSocket upgrades, so ASP.NET reads `access_token` from the query string for `/hubs` paths via `JwtBearer.OnMessageReceived`.

Q4. How do you push to one user only?

On connect, add the connection to group `user-{id}`. On assign, `SendAsync` to that group with the notification DTO.

Q5. How would you prevent duplicate SignalR connections?

Use a module singleton (or context provider) with subscribers, instead of creating a new `HubConnection` in every component effect.

Q6. What would you improve further in production?

Reconnect metrics, presence, mark-as-delivered, integration tests for assign→notify, and polling fallback if WebSockets are blocked.

7. One-minute interview pitch (say this)

"We added SignalR notifications for task assignment. In Next.js dev we hit two issues: RTK Query refetch on a skipped notification list query, and SignalR negotiate abort under React Strict Mode which Next displayed as a console error overlay. I isolated each stack in DevTools, validated the hub with authenticated negotiate, fixed the RTK toggle logic, and moved SignalR to a shared connection with quiet logging. Backend still persists Notifications and pushes ReceiveNotification to user-specific groups over JWT-authenticated hubs."

TaskFlow • Interview prep notes • SignalR + RTK Query + Next.js debugging